

Smart Lost & Found

Matching lost and found items with a vision-language model and embeddings

AI-ENG-110 Software Engineering · Spring 2026

Final Project Defense

M503 Team 8 · Topic 1

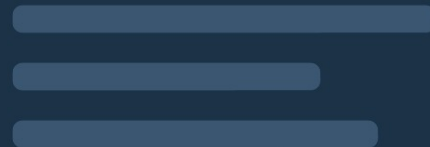
 Sadiq Sadiqov

 Ülkər Kərimova

 Oğuz Həsənlı

github.com/Oghuz20/lost-and-found-project

LOST

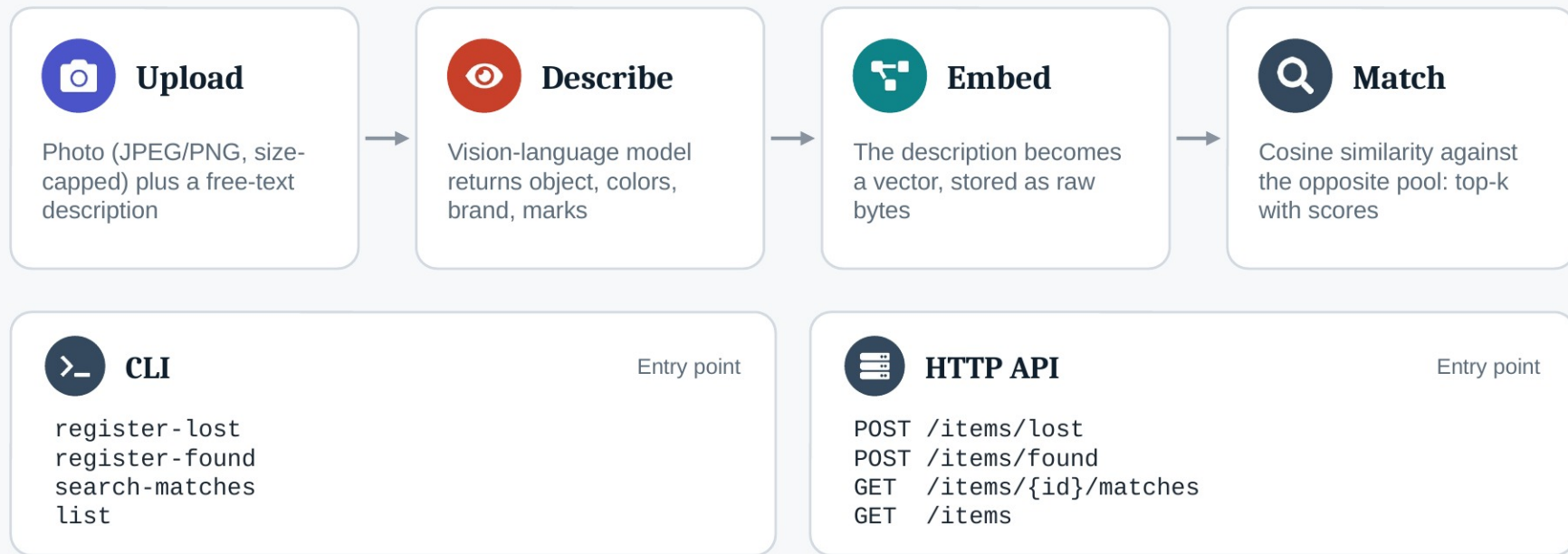


FOUND

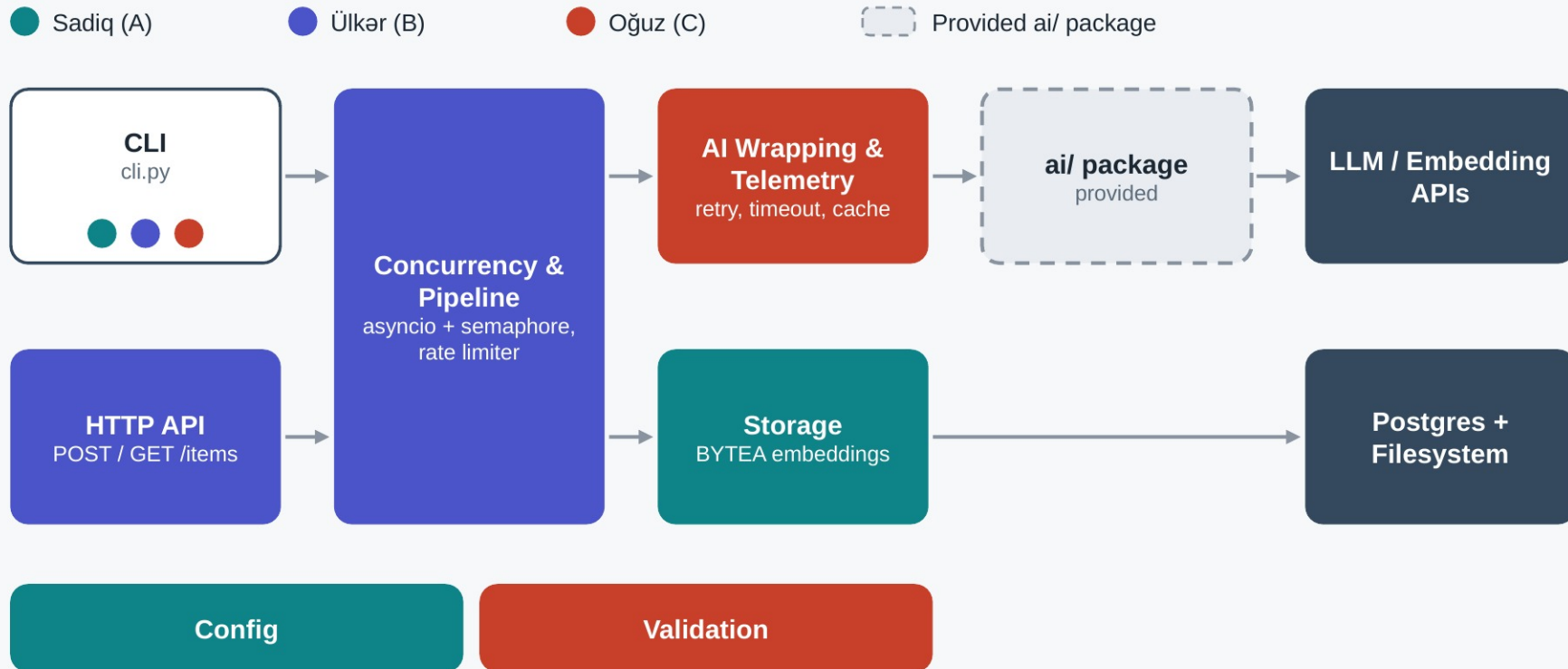


The problem in one sentence

A user uploads a photo and a description of a lost or found item. We describe it with a vision-language model, embed the description, and return the top matches from the opposite pool by cosine similarity.



Architecture



Three design decisions we can defend

1

asyncio + semaphore

over threads or processes

WHY

Every AI call is I/O-bound: a network round-trip, not CPU work. asyncio fits it without process-spawn overhead.

TRADE-OFF

ProcessPoolExecutor was rejected as unnecessary weight.

2

One custom exception: AIServiceError

over a Protocol or an error-code enum

WHY

Callers get plain try/except and never need to know which retry library sits underneath.

TRADE-OFF

Less flexible than a Protocol, but far simpler to reason about.

3

Embeddings as raw BYTEA

over JSON

WHY

Compact, with no text-encoding overhead for a numeric vector.

TRADE-OFF

We would revisit it if we ever had to query inside the vector at the SQL level.

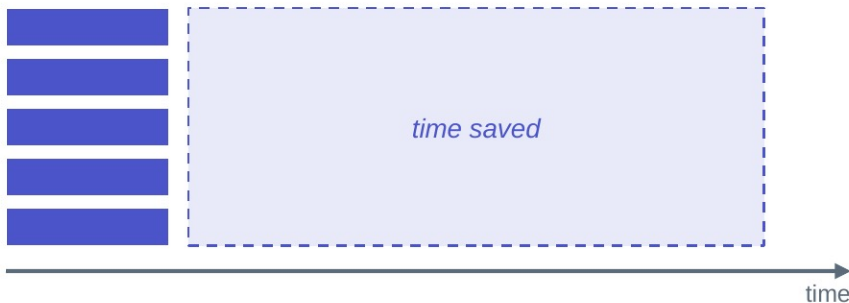
Concurrency: bounded parallelism

How the work is scheduled

Sequential



Concurrent, semaphore = 5



Each block is one AI call. The semaphore keeps at most 5 in flight. Schematic, not measured data.

What we measure

● Wall-clock time

Sequential vs concurrent, same workload

● Speedup

Gain over sequential; ideal is $N\times$

● Provider 429s

Rate-limit hits counted in every run

WHERE SPEEDUP STOPS SHORT OF $N\times$

The provider's own rate limit and network round-trip time.

`scripts/bench.py`

Robustness: one concrete failure mode

THE SCENARIO

Gemini returned 429 RESOURCE_EXHAUSTED

Not a per-minute rate limit: the daily free-tier quota (20 requests/day) ran out.

HOW WE FOUND IT

Not a synthetic test. We hit it running the demo against the real provider mid-development.



Retry

tenacity, 3 attempts, exponential backoff (1s, 2s, 4s), retrying only ProviderError.



Validate

A transient 503 on other items recovered on retry and succeeded normally.



Log

WARNING on every retry, one ERROR line at final exhaustion. Diagnosable without re-running.



Degrade

The item that exhausted its retries was excluded from matching; every other item in the batch still completed and matched.

Testing

88.59%

Total coverage

27 / 27

Provided ai/ smoke tests passing

Clean

mypy on our own code, with documented exclusions for code we don't own

Isolation: AI module mocked (FakeVLM / FakeEmbedder); HTTP layer mocked via dependency overrides

Tooling: pytest · pytest-asyncio · pytest-cov · unittest.mock

Three tests worth calling out

END-TO-END

Happy-path end-to-end match

The full flow from registration to a ranked match.

CAUGHT A BUG

Concurrency test

Caught BatchRunner discarding an entire batch on one failure.

CAUGHT A BUG

Error-path test

Caught an unhandled-exception leak, fixed with a final catch-all.

Limitations & what we'd do next



Brittle today

1

Retry treats every 429 alike

A per-minute rate limit and a per-day quota get the same policy, so retrying an exhausted quota wastes what remains.

2

Thread-based timeouts

Python cannot forcibly kill an in-flight thread.

3

Unbounded embedding cache

No eviction policy, so memory grows on a long-running process.



Given another week

1

Classify provider failures

By type, and skip retrying quota exhaustion specifically.

2

Load-test the rate limiter

Under sustained, real concurrent traffic.

3

Invalidate stale embeddings

Detect when the embedding model changes and invalidate what is stored.

Team contributions



Sadiq Sadiqov

- Config
- Storage (Postgres, filesystem)
- Dockerfile
- CI



Ülkər Kərimova

- HTTP API
- Concurrency
- Rate limiter
- Benchmark



Oğuz Həsənlı

- AI wrapping (retry, timeout, cache)
- Telemetry
- Tests
- Report

WE CAN ALL ANSWER

Any of us can walk through any module on demand. Pick a file and we'll defend it.

AI tool use

Claude was used across all three parts, most substantially for AI wrapping and telemetry. Report §9 has the full disclosure, including bugs it introduced that we caught and had fixed: a fake timeout, a silent exception leak, and a batch-runner bug that discarded results on partial failure.

Questions?

We can walk through any file in the repo.

M503 Team 8 · Smart Lost & Found

github.com/Oghuz20/lost-and-found-project

